

ZOZOTOWNリプレイスでの Skills導入までの流れとこれから

株式会社ZOZO

ZOZOTOWN開発本部 ZOZOTOWN開発3部 バックエンドリプレイスブロック

平林 雅士

Copyright © ZOZO, Inc.





株式会社ZOZO

ZOZOTOWN開発本部 ZOZOTOWN開発3部

バックエンドリプレイスブロック

ばや (@bayacollector)

ZOZOTOWNリプレイスプロジェクトでBFF (Backend For Frontend) を担当。Claude CodeやSkillsを活用した開発の推進に取り組んでいます。



ZOZOTOWN

<https://zozo.jp/>

- ファッションEC
- 1,700以上のショップ、11,000以上のブランドの取り扱い
- 常時107万点以上の商品アイテム数と毎日平均2,700点以上の新着商品を掲載（2025年12月末時点）
- ブランド古着のファッションゾーン「ZOZOUSED」やコスメ専門モール「ZOZOCOSME」、シューズ専門ゾーン「ZOZOSHOES」、ラグジュアリー&デザイナーズゾーン「ZOZOVILLA」を展開
- 即日配送サービス
- ギフトラッピングサービス
- ツケ払い など



アジェンダ

- 今日話したいこと
 - Skills導入の目的とゴール
- これまでの取り組み
 - 業務の標準化からSkills化までのステップ
- これからの取り組み
 - レビューと品質保証の両立



今日話したいこと

今の皆さんの業務で何をSkillsにしていくとよいかを、私たちの知見を通じて Step by stepで知ってもらうこと。

Skillsの効果的な利用方法は「いきなりSkillsを書く」ことではなかった。業務の標準化・テンプレート化を積み重ねてきた結果、自然とSkills化できる状態になった、というストーリーをお伝えしたい。

これまでの取り組み：現状の共有

Skillsを活用した仕様駆動開発を行っている

仕様書をClaude Codeに書かせ、レビューを経てソースコードを自動生成するフロー。仕様書の品質がコードの品質に直結するため、仕様のテンプレートと進め方をSkillsとして定義している。

ここに至るまでに3つのステップがあった。



Skills導入までの3ステップ

Step1

2025年 上旬

Claude Code
導入以前

ガイドライン整備
テンプレート作成
人間のための標準化



Step2

2025年 中旬

Claude Code
導入中

業務のStep化
入出力の明文化
AIへの手順書の原型



Step3

2025年 下旬

Skills化
業務再構築

テンプレ→reference
進め方→SKILL.md
仕様駆動開発の確立

Step 1a: 開発におけるガイドライン整備 (導入以前)

属人化を排除し、開発品質を統一するためにガイドラインを整備
レビューで指摘するポイントとして、社内における明確な基準ができた

アーキテクチャ
ガイドライン

ブランチ戦略
ガイドライン

エラーハンドリング
ガイドライン

API
ガイドライン

→ 開発品質が統一され、属人化が解消

Step 1b: 業務フローのテンプレート整備 (導入以前)

既存調査書とAPI開発設計書のテンプレートを整備

既存調査書
テンプレート

API開発設計書
テンプレート

→ テンプレート化により「AIへの手順書 Skills」の原型が生まれた

Step 1b: 既存調査書テンプレート (導入以前)

リプレイス前のVBScriptで書かれている処理を体系化する
膨大なソースコードの中から、どの部分をPickupしてまとめるかを整理した資料
主にリプレイスする部分はBFFなのでフロント, 基盤とのIFを重視して記載

1. フロントエンドとのIF
 - a. 型やパラメータの列挙
 - b. リクエストパラメータのバリデーション
 - c. エラーハンドリング
2. 基盤とのIF
 - a. どのマイクロサービスに接続しているか (商品, 検索, 会員 etc)
 - b. キャッシュ時間
 - c. エラーハンドリング
3. 後々AIで読み込めるように新環境のGitHub Repositoryに旧コードをコピペ

詳しい構成などはTech Blogにて

Agent Skills導入で既存コード調査のリードタイムを2～5日から数時間へ短縮



Step 1b: API開発設計書テンプレート (導入以前)

新環境における設計資料

いわゆる Design Doc

フォーマットとしてはほとんど既存調査と変わらない

既存調査結果からリプレイス不要なものをなどを削ったりして正規化された資料

シーケンス図なども記載することで全体感をイメージしやすいようになる

→ これをベースの「仕様」として仕様駆動開発を進めていった



Step 2: 業務をステップ化 (導入中)

- まだSkillsという概念がない頃
- Context Engineeringのような、業務を小さい単位に分割することが良しとされた
- API開発の進め方を明確なステップに分解



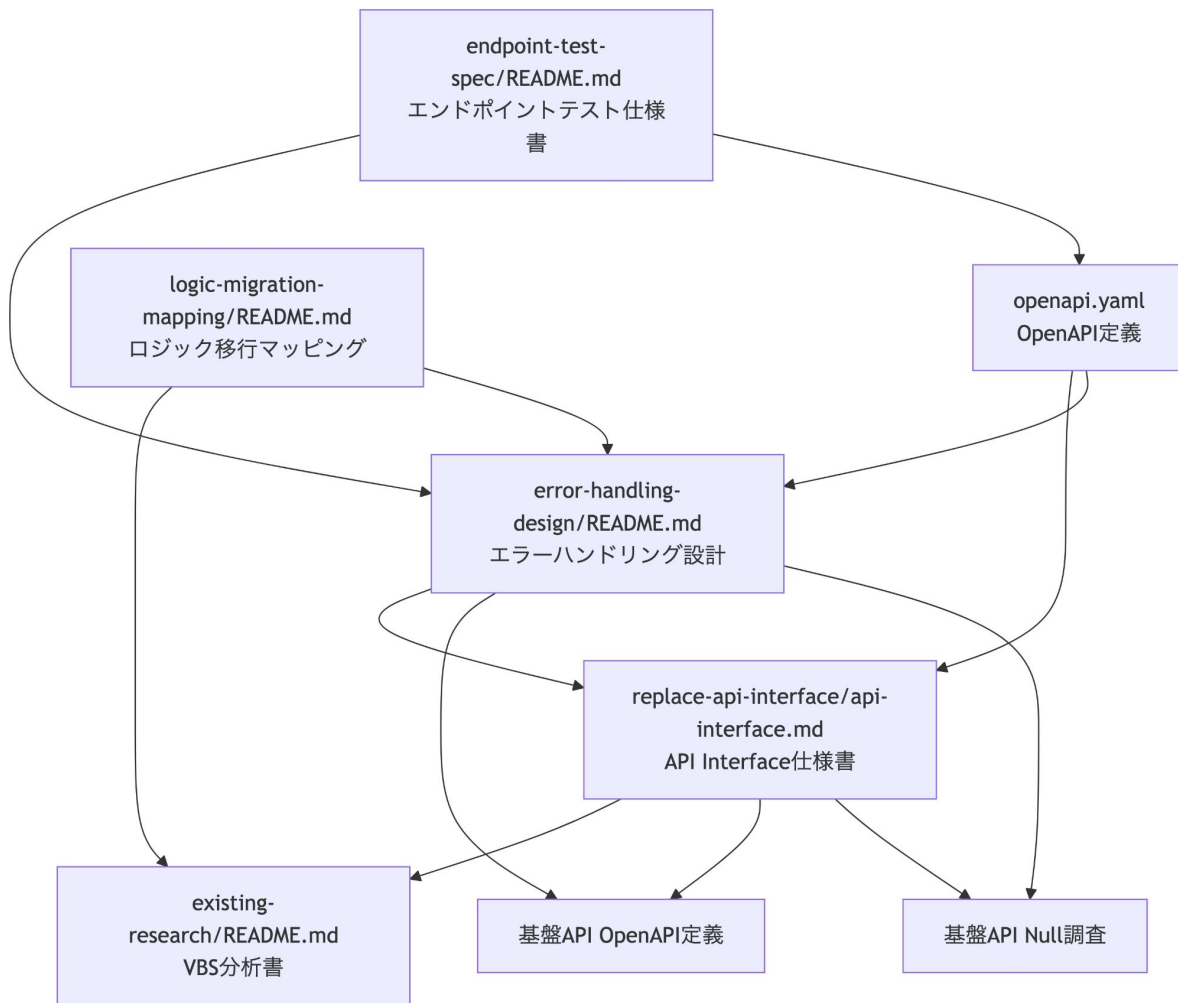
→ AIに渡す「仕様書」の原型ができた

Step 3: 各テンプレートのSkills化 (業務再構築)

- 各ステップをある程度まとめた単位でのSkillsに分解
- 設計書の依存関係を整理して、より細かい粒度での仕様策定を実施
- ドメインモデリングの考え方と一緒なので、使う頭は開発とあまり変わらない。

→仕様駆動開発のフロー確立





Skills化の効果と課題（業務再構築）

効果：仕様書作成が爆速に

テンプレートに沿った仕様書をClaude Codeが数分で生成。

課題①：手戻りをチェックする手間が多い

レビュイー（開発者）がレビューを行うのが大変

シンプルに一回に生成される文章量が多い & ハルシネーションを見分けることが困難

課題②：レビューが追いつかない

生成速度が上がった分、レビュー待ちがボトルネックに。AIが書いた仕様書を別の形で品質担保していく必要がある。

これからの取り組み: すべてのステップをAIで完結させる

一連のステップを1つのコマンド実行だけで完結できるようにしていく。
事前準備としては整っているので、後はワークフロー設計と改善ループを回すことに注力する。

→ いわゆるハーネスエンジニアリングを構想・実践中



品質保証をハーネスで拡充

"The generator-evaluator loop maps naturally onto the software development lifecycle, where code review and QA serve the same structural role as the design evaluator."

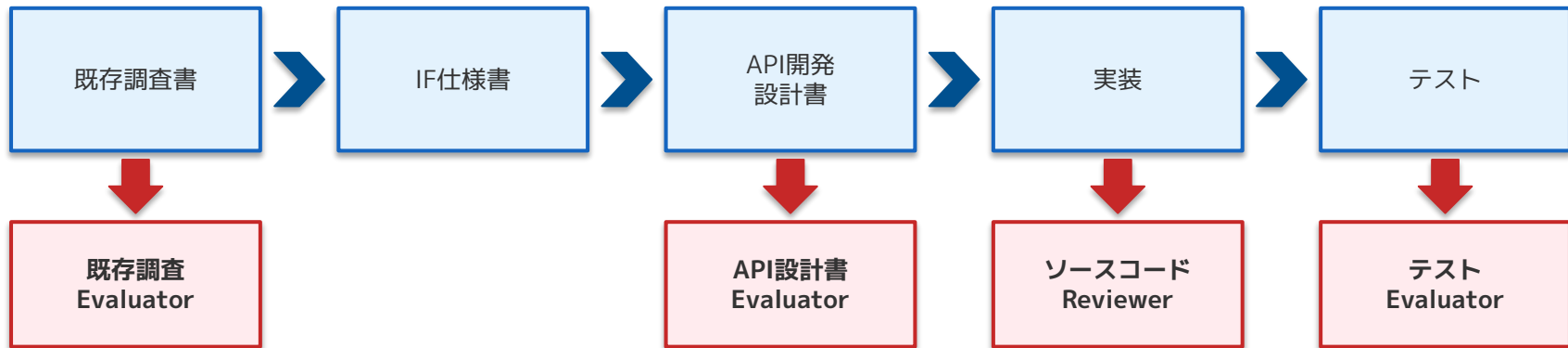
— Anthropic Engineering Blog: Harness Design for Long-Running Apps



→ 生成と評価を分離し、品質ゲートを自動化する

取り組み1: 各ステップでのEvaluator Agent導入

Generator (仕様書作成 Skills)



Evaluator (レビュー Skills)

不合格 → 具体的なfeedbackと共にGeneratorへ差し戻し → 再生成

→ 各工程にEvaluatorを配置し、人間はドメイン判断・設計判断に集中

ハーネス化で生まれる新たな課題

開発スピードが上がりすぎて、シニアメンバーが全体を見きれなくなる

知識を溜め込むリードタイムが大幅に短縮
→ 「横に広く強い人が全部見る」が事実上不可能に（既に発生中）

→ 実装者自身がdeep diveし、機能を追求する必要がある

ただし、シニアの視点が欠けたままリリースするリスクが残る

→ 内部品質レビュー + 外部品質レビュー の2段階レビューで担保する

形式: 同期MTG (約1h) / レビュイー1名 + 複数レビュワー / 成果物に対してあらゆる角度から質問

取り組み2: 2段階の同期的な品質レビュー

内部品質レビュー

アプリケーション単体

IF設計やコードの意図を深堀り

どの角度からの質問にも答えられる
状態を求める

「AIが作ったから」をなくす
→ レビューーが誰よりも詳しい状態を維持

目的: レビューーの当事者性の醸成、教育

外部品質レビュー

プロダクション全体

テスト手法とその妥当性をレビュー

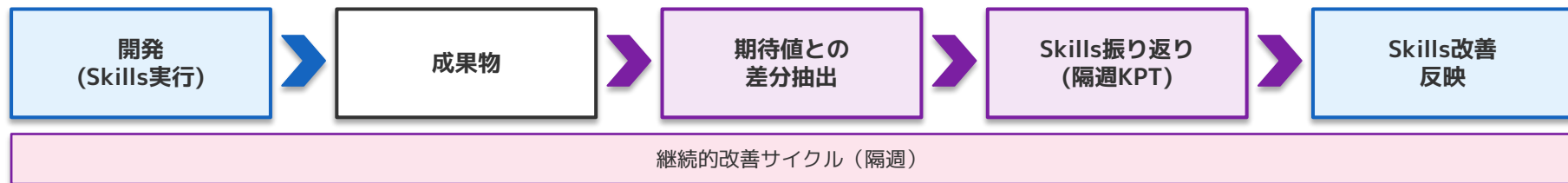
テスト観点・ケースの網羅性
負荷試験・障害試験の要否判断

ソフトウェアとしてプロダクションレディかどうか
を見極める

目的: リリース品質の客観的担保、教育

取り組み3: Skills 改善振り返り

隔週KPT — 成果物と期待値の差分をSkillsに反映する



各メンバーが改善点を持ち寄る

実際の開発で気づいた「Skillsがこう動いてほしいかった」を共有

成果物と期待値のギャップを特定

生成された仕様書・コードの品質が基準に達していない箇所を洗い出し

SKILL.md / referenceを更新

改善点をSkillsに即時反映 → 次の開発サイクルで品質が上がる

→ Skillsが「チームの集合知」として継続的に進化する

まとめ

ZOZOTOWNリプレイスプロジェクトで行ってきたこと
業務の標準化 → ステップ化 → Skills化 → 仕様駆動開発 → ハーネス化

Skillsは銀の弾丸ではない。

標準化・仕組み化への地道な投資が、後のSkills化に効いた。

最終的なハーネスエンジニアリングも一つの標準化から始まる気がする。





ZOZO